

Desenvolvimento WEB

Professor: Janilson

Curso: BSI

SISTEMA DE CONTROLE DE VERSÃO DE CÓDIGO

É UM SISTEMA QUE PERMITE REGISTRAR MUDANÇAS REALIZADAS NO CÓDIGO E MANTÊ-LAS ORGANIZADAS E FACILMENTE IDENTIFICÁVEIS.

É USADO PRINCIPALMENTE POR DESENVOLVEDORES PARA GERENCIAR PROJETOS DE SOFTWARE, MAS TAMBÉM PODE SER USADO PARA OUTROS TIPOS DE ARQUIVOS, COMO TEXTOS, IMAGENS, ENTRE OUTROS.

O GIT MANTÉM UMA HISTÓRIA COMPLETA DE TODAS AS ALTERAÇÕES REALIZADAS NO CÓDIGO, TORNANDO FÁCIL VOLTAR A VERSÕES ANTERIORES OU COMPARAR DIFERENÇAS ENTRE VERSÕES.

ALÉM DISSO, O GIT É UM SISTEMA DISTRIBUÍDO, O QUE SIGNIFICA QUE O CÓDIGO PODE SER FACILMENTE COMPARTILHADO E COLABORADO COM OUTROS DESENVOLVEDORES.

SEM GIT



Projeto v.1



Projeto v.2



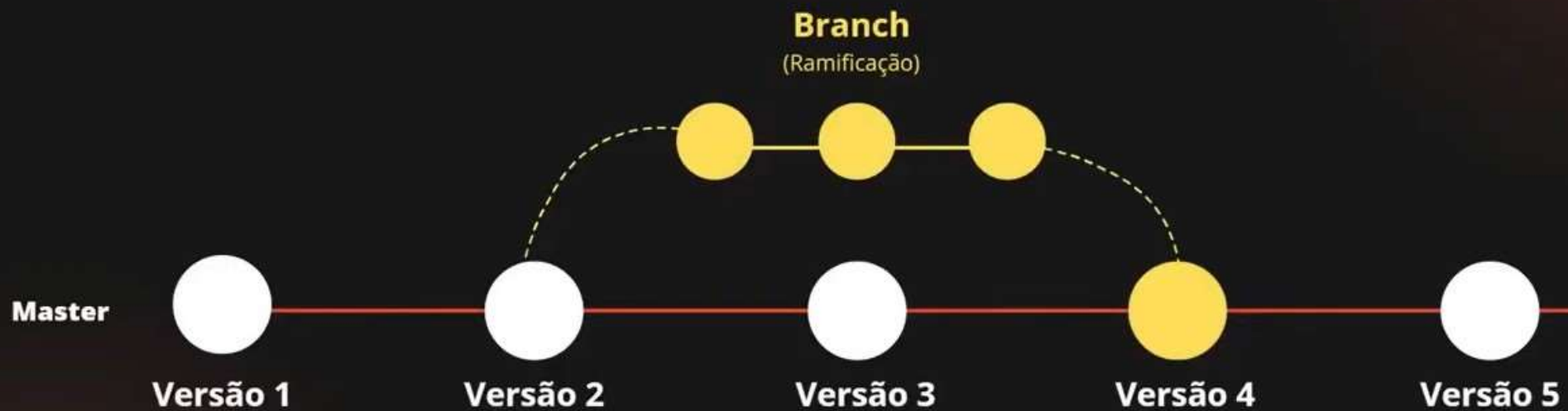
Projeto v.3

COM GIT

Commits
(Commitar)



COM GIT



- **Capítulo 1 – Introdução ao Git e GitHub**
- **Git:** sistema de controle de versão distribuído, criado por Linus Torvalds em 2005.
- **GitHub:** plataforma online que hospeda repositórios Git, permitindo colaboração entre desenvolvedores.
- **Importância:**
 - Mantém histórico completo de alterações.
 - Facilita voltar a versões anteriores.
 - Permite trabalho colaborativo em equipe.

- **Capítulo 2 – Principais Comandos do Git**
- **git init:** inicia um novo repositório local.
- **git add:** adiciona arquivos à área de *stage*.
- **git commit:** grava alterações como uma nova versão.
- **git status:** mostra o status atual do repositório.
- **git log:** exibe histórico de commits.

- **git branch:** lista branches existentes.
- **git checkout:** alterna entre branches ou versões.
- **git merge:** une branches diferentes.
- **git remote add origin:** conecta repositório local ao GitHub.
- **git push:** envia alterações para o GitHub.
- **git clone:** baixa repositório do GitHub para a máquina local.

- **Capítulo 3 – Conceitos Fundamentais**
- **Branch (ramificação):** linha de desenvolvimento paralela.
- **Commit:** registro de uma alteração no código.
- **HEAD:** ponteiro que indica a versão atual.
- **Master/Main:** branch principal do projeto.
- **Stage:** área intermediária antes do commit.

- **Capítulo 4 – GitHub na Prática**
- **Repositórios públicos e privados.**
- **Colaboração:** equipes acessam e contribuem em tempo real.
- **Pull Requests:** sugerem alterações para revisão.
- **Issues:** registram problemas ou melhorias.
- **Forks:** cópia de um repositório para desenvolvimento independente.

- **Capítulo 5 – Exercícios Resolvidos**

- **Exemplo:**

- Criar um repositório local:
- `git init`
- `git add .`
- `git commit -m "Versão inicial"`
- Conectar ao GitHub:
- `git remote add origin`
`https://github.com/usuario/repositorio.git`
- `git push -u origin master`

- **Exemplo de branch:**
- `git checkout -b dev`
- `git add .`
- `git commit -m "Nova funcionalidade"`
- `git checkout master`
- `git merge dev`

- **Capítulo 6 – Exercícios Propostos**
- Crie um repositório local e faça três commits diferentes (inicial, alteração de layout, inclusão de menu).
- Crie uma branch chamada *feature-login*, adicione um arquivo login.html e depois faça o merge com a branch principal.
- Explique a diferença entre git clone e git pull.
- Configure um repositório remoto no GitHub e envie alterações usando git push.

